we ablate these options and find that this normalization is required to train the recurrence at scale[2].

Given an embedding matrix E and embedding scale γ, the prelude block first embeds input tokens $\mathbf{x}$ as $\gamma E(\mathbf{x})$, and then to applies l_P many prelude layers with the layout described above.

Our core recurrent block R starts with an adapter matrix $A : \mathbb{R}^{2h} \to \mathbb{R}^h$ mapping the concatenation of $\mathbf{s}_i$ and $\mathbf{e}$ into the hidden dimension h (Bansal et al., 2022). While re-incorporation of initial embedding features via addition rather than concatenation works equally well for smaller models, we find that concatenation works best at scale. This is then fed into l_R transformer layers. At the end of the core block the output is again rescaled with an RMSNorm n_c.

The coda contains l_C layers, normalization by n_c, and projection into the vocabulary using tied embeddings E^T.

In summary, we can summarize the architecture by the triplet (l_P, l_R, l_C), describing the number of layers in each stage, and by the number of recurrences r, which may vary in each forward pass. We train a number of small-scale models with shape $(1, 4, 1)$ and hidden size $h = 1024$, in addition to a large model with shape $(2, 4, 2)$ and $h = 5280$. This model has only 8 "real" layers, but when the recurrent block is iterated, e.g. 32 times, it unfolds to an effective depth of $2 + 4r + 2 = 132$ layers, constructing computation chains that can be deeper than even the largest fixed-depth transformers (Levine et al., 2021; Merrill et al., 2022).

3.3. Training Objective

Training Recurrent Models through Unrolling. To ensure that the model can function when we scale up recurrent iterations at test-time, we randomly sample iteration counts during training, assigning a random number of iterations r to every input sequence (Schwarzschild et al., 2021b). We optimize the expectation of the loss function L over random samples x from distribution X and random iteration counts r from distribution Λ.

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}\in X}\mathbb{E}_{r\sim\Lambda} L\left(m_\theta(\mathbf{x}, r), \mathbf{x}'\right).$$

Here, m represents the model output, and $\mathbf{x}'$ is the sequence $\mathbf{x}$ shifted left, i.e., the next tokens in the sequence $\mathbf{x}$. We choose Λ to be a *log-normal Poisson distribution*. Given a targeted mean recurrence $\bar{r} + 1$ and a variance that we set to $\sigma = \frac{1}{2}$, we can sample from this distribution via

$$\tau \sim \mathcal{N}(\log(\bar{r}) - \frac{1}{2}\sigma^2, \sigma) \tag{1}$$

$$r \sim \mathcal{P}(e^\tau) + 1, \tag{2}$$

[2]Note also that technically n_3 is superfluous, but we report here the exact norm setup with which we trained the final model.

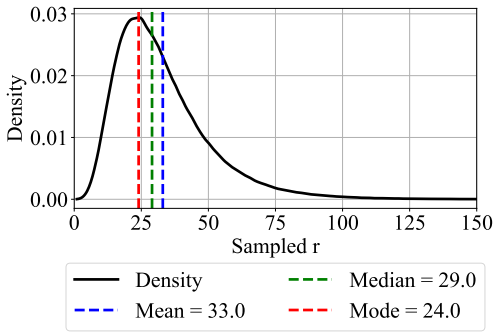

Figure 3: We use a log-normal Poisson Distribution to sample the number of recurrent iterations for each training step.

given the normal distribution $\mathcal{N}$ and Poisson distribution $\mathcal{P}$, see Figure 3. The distribution most often samples values less than $\bar{r}$, but it contains a heavy tail of occasional events in which significantly more iterations are taken.

Truncated Backpropagation. To keep computation and memory low at train time, we backpropagate through only the last k iterations of the recurrent unit. This enables us to train with the heavy-tailed Poisson distribution Λ, as maximum activation memory and backward compute is now independent of r. We fix $k = 8$ in our main experiments. At small scale, this works as well as sampling k uniformly, but with set fixed, the overall memory usage in each step of training is equal. Note that the prelude block still receives gradient updates in every step, as its output $\mathbf{e}$ is injected in every step. This setup resembles truncated backpropagation through time, as commonly done with RNNs, although our setup is recurrent in depth rather than time (Williams and Peng, 1990; Mikolov et al., 2011).

4. Training a large-scale recurrent-depth Language Model

After verifying that we can reliably train small test models up to 10B tokens, we move on to larger-scale runs. Given our limited compute budget, we could either train multiple tiny models too small to show emergent effects or scaling, or train a single medium-scale model. Based on this, we prepared for a single run, which we detail below.

4.1. Training Setup

We describe the training setup, separated into architecture, optimization setup and pretraining data. We publicly release all training data, pretraining code, and a selection of intermediate model checkpoints.

Pretraining Data. Given access to only enough compute for a single large scale model run, we opted for a dataset mixture that maximized the potential for emergent reasoning behaviors, not necessarily for optimal benchmark per-

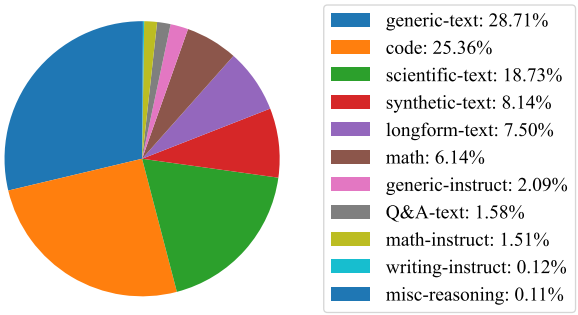

Figure 4: Distribution of data sources that are included during training. The majority of our data is comprised of generic webtext, scientific writing and code.

formance. Our final mixture is heavily skewed towards code and mathematical reasoning data with (hopefully) just enough general webtext to allow the model to acquire standard language modeling abilities. All sources are publicly available. We provide an overview in Figure 4. Following Allen-Zhu and Li (2024), we directly mix relevant instruction data into the pretraining data. However, due to compute and time constraints, we were not able to ablate this mixture. We expect that a more careful data preparation could further improve the model's performance. We list all data sources in Appendix C.

Tokenization and Packing Details. We construct a vocabulary of 65536 tokens via BPE (Sennrich et al., 2016), using the implementation of Dagan (2024). In comparison to conventional tokenizer training, we construct our tokenizer directly on the instruction data split of our pretraining corpus, to maximize tokenization efficiency on the target domain. We also substantially modify the pre-tokenization regex (e.g. of Dagan et al. (2024)) to better support code, contractions and LaTeX. We include a `<|begin_text|>` token at the start of every document. After tokenizing our pretraining corpus, we pack our tokenized documents into sequences of length 4096. When packing, we discard document ends that would otherwise lack previous context, to fix an issue described as the "grounding problem" in Ding et al. (2024), aside from several long-document sources of mathematical content, which we preserve in their entirety.

Architecture and Initialization. We scale the architecture described in Section 3, setting the layers to $(2, 4, 2)$, and train with a mean recurrence value of $\bar{r} = 32$. We mainly scale by increasing the hidden size to $h = 5280$, which yields 55 heads of size of 96. The MLP inner dimension is 17920 and the RMSNorm ε is 10^{-6}. Overall this model shape has about 1.5B parameters in non-recurrent prelude and head, 1.5B parameters in the core recurrent block, and 0.5B in the tied input embedding.

At small scales, most sensible initialization schemes work. However, at larger scales, we use the initialization of Takase et al. (2024) which prescribes a variance of $\sigma_h^2 = \frac{2}{5h}$. We initialize all parameters from a truncated normal distribution (truncated at 3σ) with this variance, except all out-projection layers, where the variance is set to $\sigma_{\text{out}}^2 = \frac{1}{5hl}$, for $l = l_P + \bar{r} l_R + l_C$ the number of effective layers, which is 132 for this model. As a result, the out-projection layers are initialized with fairly small values (Goyal et al., 2018). The output of the embedding layer is scaled by $\sqrt{h}$. To match this initialization, the state s_0 is also sampled from a truncated normal distribution, here with variance $\sigma_s^2 = \frac{2}{5}$.

Locked-Step Sampling. To enable synchronization between parallel workers, we sample a single depth r for each micro-batch of training, which we synchronize across workers (otherwise workers would idle while waiting for the model with the largest r to complete its backward pass). We verify at small scale that this modification improves compute utilization without impacting convergence speed, but note that at large batch sizes, training could be further improved by optimally sampling and scheduling independent steps r on each worker, to more faithfully model the expectation over steps in Equation (1).

Optimizer and Learning Rate Schedule. We train using the Adam optimizer with decoupled weight regularization ($\beta_1 = 0.9$, $\beta_2 = 0.95$, $\eta = 5 \times 10^{-4}$) (Kingma and Ba, 2015; Loshchilov and Hutter, 2017), modified to include update clipping (Wortsman et al., 2023b) and removal of the ε constant as in Everett et al. (2024). We clip gradients above 1. We train with warm-up and a constant learning rate (Zhai et al., 2022; Geiping and Goldstein, 2023), warming up to our maximal learning rate within the first 4096 steps.

4.2. Compute Setup and Hardware

We train this model using compute time allocated on the Oak Ridge National Laboratory's *Frontier* supercomputer. This HPE Cray system contains 9408 compute nodes with AMD MI250X GPUs, connected via 4xHPE Slingshot-11 NICs. The scheduling system is orchestrated through SLURM. We train in bfloat16 mixed precision using a PyTorch-based implementation (Zamirai et al., 2021).

Device Speed and Parallelization Strategy. Nominally, each MI250X chip[3] achieves 192 TFLOP per GPU (AMD, 2021). For a single matrix multiplication, we measure a maximum achievable speed on these GPUs of 125 TFLOP/s on our software stack (ROCM 6.2.0, PyTorch 2.6 prerelease 11/02) (Bekman, 2023). Our implementation, using extensive PyTorch compilation and optimization of the hidden dimension to $h = 5280$ achieves a single-node training

[3]Technically, each node contains 4 dual-chip MI250X cards, but its main software stack (ROCm runtime) treats these chips as fully independent.

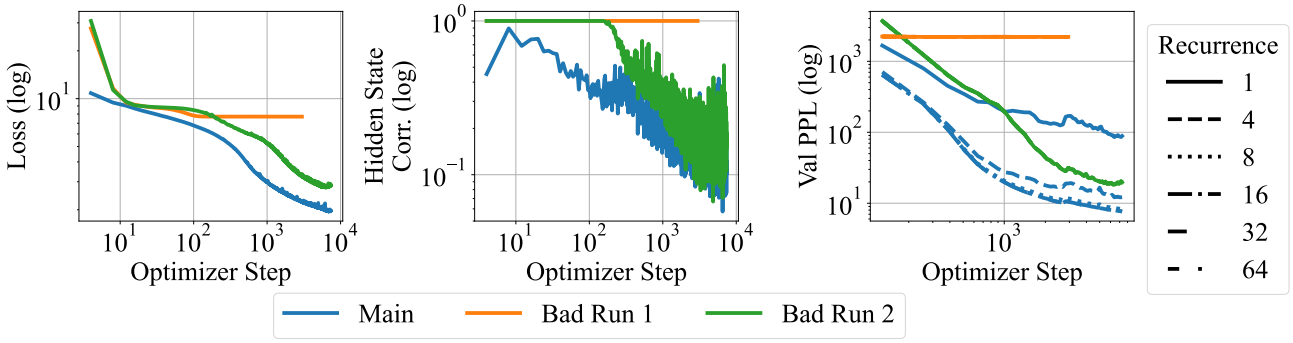

Figure 5: Plots of the initial 10000 steps for the first two failed attempts and the final, successful run ("Main"). Note the hidden state collapse (middle) and collapse of the recurrence (right) in the first two failed runs, underlining the importance of our architecture and initialization in inducing a recurrent model and explain the underperformance of these runs in terms of pretraining loss (left).

speed of 108.75 TFLOP/s, i.e. 87% AFU ("Achievable Flop Utilization"). Due to the weight sharing inherent in our recurrent design, even our largest model is still small enough to be trained using only data (not tensor) parallelism, with only optimizer sharding (Rajbhandari et al., 2020) and gradient checkpointing on a per-iteration granularity. With a batch size of 1 per GPU we end up with a global batch size of 16M tokens per step, minimizing inter-GPU communication bandwidth.

When we run at scale on 4096 GPUs, we achieve 52-64 TFLOP/s per GPU, i.e. 41%-51% AFU, or 1-1.2M tokens per second. To achieve this, we wrote a hand-crafted distributed data parallel implementation to circumvent a critical AMD interconnect issue, which we describe in more detail in Appendix A.2. Overall, we believe this may be the largest language model training run to completion in terms of number of devices used in parallel on an AMD cluster, as of time of writing.

Training Timeline. Training proceeded through 21 segments of up to 12 hours, which scheduled on Frontier mostly in early December 2024. We also ran a baseline comparison, where we train the same architecture but in a feedforward manner with only 1 pass through the core/recurrent block. This trained with the same setup for 180B tokens on 256 nodes with a batch size of 2 per GPU. Ultimately, we were able to schedule 795B tokens of pretraining of the main model. Due to our constant learning rate schedule, we were able to add additional segments "on-demand", when an allocation happened to be available.

4.3. Importance of Norms and Initializations at Scale

At small scales all normalization strategies worked, and we observed only tiny differences between initializations. The same was not true at scale. The first training run we started was set up with the same block sandwich structure as described above, but parameter-free RMSNorm layers, no embedding scale γ, a parameter-free adapter $A(\mathbf{s}, \mathbf{e}) = \mathbf{s} + \mathbf{e}$, and a peak learning rate of 4×10^{-4}. As shown in Figure 5, this run ("Bad Run 1", orange), quickly stalled.

While the run obviously stopped improving in training loss (left plot), we find that this stall is due to the model's representation collapsing (Noci et al., 2022). The correlation of hidden states in the token dimension quickly goes to 1.0 (middle plot), meaning the model predicts the same hidden state for every token in the sequence. We find that this is an initialization issue that arises due to the recurrence operation. Every iteration of the recurrence block increases token correlation, mixing the sequence until collapse.

We attempt to fix this by introducing the embedding scale factor, switching back to a conventional pre-normalization block, and switching to the learned adapter. Initially, these changes appear to remedy the issue. Even though token correlation shoots close to 1.0 at the start ("Bad Run 2", green), the model recovers after the first 150 steps. However, we quickly find that this training run is not able to leverage test-time compute effectively (right plot), as validation perplexity is the same whether 1 or 32 recurrences are used. This initialization and norm setup has led to a local minimum as the model has learned early to ignore the incoming state $\mathbf{s}$, preventing further improvements.

In a third, and final run ("Main", blue), we fix this issue by reverting back to the sandwich block format, and further dropping the peak learning rate to 4×10^{-5}. This run starts smoothly, never reaches a token correlation close to 1.0, and quickly overtakes the previous run by utilizing the recurrence and improving with more iterations.

With our successful configuration, training continues smoothly for the next 750B tokens without notable interruptions or loss spikes. We plot training loss and perplexity at different recurrence steps in Figure 6. In our material, we refer to the final checkpoint of this run as our "main model", which we denote as *Huginn-0125*[4].

[4] /huː gɪn/, transl. "thought", is a raven depicted in Norse mythology. Corvids are surprisingly intelligent for their size, and and of course, as birds, able to unfold their wings at test-time.